

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 0 — ENVIRONMENT SETUP

For: Python beginners who have never used an LLM API, LangChain, or a local model.

Targets: LangChain 1.x · langchain-google-genai 4.x · langchain-ollama · Python 3.10+

Estimated time for this phase: 45–75 minutes · **Cost:** Free

The Full Course Map

This is a 13-phase journey. Each phase ships as its own PDF. You are reading **Phase 0**. By the end you'll be able to build chains, a RAG pipeline, and a tool-using agent — in both a cloud (Gemini) and a local (Ollama) setup.

Ph 0 **Environment Setup** ← you are here

Python env, packages, Gemini API key, Ollama + local models, secrets in .env, smoke test.

Ph 1 **LangChain Fundamentals**

What LangChain is/isn't, the Runnable interface, LCEL and the `|` pipe operator.

Ph 2 **Working with Models**

Chat models vs LLMs, invoke / stream / batch, temperature and other parameters.

Ph 3 **Prompts & Output Parsers**

Prompt templates, message roles, few-shot prompting, structured output with Pydantic.

Ph 4 **Chains & LCEL Composition**

Sequencing, parallel branches, conditional routing, custom functions, debugging.

Ph 5 **Memory & Conversational Apps**

Adding chat history (modern LangGraph persistence — not the legacy memory classes).

Ph 6 **Loading & Splitting Documents**

Loaders for PDFs/text/web, chunking strategies.

Ph 7 **Embeddings & Vector Stores**

Embeddings with Gemini and local Ollama, storing/querying a FAISS vector DB.

Ph 8 **Building a RAG Application**

The full retrieval-augmented generation pipeline, end to end.

Ph 9 **Tools & Agents**

Custom tools, tool-calling, the modern `create_agent` API (not legacy AgentExecutor).

Ph 10 **LangGraph Basics**

Stateful multi-step workflows; a simple agent graph.

Ph 11 **Observability & Deployment**

LangSmith tracing overview; wrapping a chain in a simple API.

Ph 12 **Capstone Project**

A document Q&A assistant with memory, RAG, and a tool — swappable Gemini/Ollama backends.

Phase 0 · Environment Setup

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Run Python in an isolated environment, install every package this course needs, hold a live Gemini API key, run a local model through Ollama, keep your secrets out of your code, and prove both backends work with a tiny test script.

In this phase

1. 0.1 — A few words before we touch anything (key terms)
2. 0.2 — Check your Python (3.10+)
3. 0.3 — Create a project folder and a virtual environment
4. 0.4 — Install the course packages
5. 0.5 — Get a free Google Gemini API key
6. 0.6 — Install Ollama and pull local models
7. 0.7 — Store secrets safely in a `.env` file
8. 0.8 — Smoke test: prove Gemini works
9. 0.9 — Smoke test: prove Ollama works
10. 0.10 — Common errors & troubleshooting
11. 0.11 — Mini practice exercise
12. Terms introduced this phase (mini-glossary)

0.1 — A few words before we touch anything

This whole course teaches you to build apps on top of **Large Language Models**. Before we install anything, let's define the words you'll see on every page from now on. Don't memorise them — just read once.

KEY TERMS

- **LLM (Large Language Model):** a program trained on huge amounts of text that, given some input text, predicts useful output text. ChatGPT, Gemini, and Llama are LLMs. You give it words; it gives you words back.
- **Token:** the unit an LLM actually reads and writes. Roughly $\frac{3}{4}$ of a word in English — "LangChain" might be 2–3 tokens. Providers measure usage (and sometimes cost) in tokens.
- **Inference:** the act of running a model to get an answer. "Calling the model" and "running inference" mean the same thing.
- **API key:** a secret password-like string that proves *you* are allowed to use a cloud service (like Google's Gemini). You send it with every request. Anyone who has it can spend on your account — so you keep it secret.

- **Cloud model vs local model:** a **cloud** model (Gemini) runs on Google's computers; you call it over the internet with an API key. A **local** model (via Ollama) runs on *your own machine* — no internet, no key, no per-use cost, but limited by your hardware.
- **LangChain:** a Python library that gives all these models one common interface and lets you wire them together with prompts, data, and tools. We'll define it properly in Phase 1.

WHY TWO BACKENDS?

This course uses **both** Gemini (cloud) and Ollama (local) for every concept. Gemini is fast, capable, and free to start, but needs internet and a key. Ollama is private, free forever, and works offline, but is slower and weaker on a typical laptop. Learning both means you can pick the right tool later — and LangChain makes them almost interchangeable in code.

0.2 — Check your Python (3.10+)

LangChain 1.x requires **Python 3.10 or newer**. Open a terminal (macOS/Linux: Terminal; Windows: PowerShell) and run:

```
python3 --version
```

You should see something like `Python 3.12.3`. If the number is 3.10 or higher, you're set.

IF THAT DIDN'T WORK

- **command not found / wrong version:** install Python from python.org/downloads, then reopen the terminal.
- **On Windows, `python` opens the Microsoft Store:** use `py --version` instead, and replace `python3` with `py` throughout.
- **You see Python 3.9 or lower:** you must upgrade — LangChain 1.x will not install on it.

0.3 — Create a project folder and a virtual environment

TERM: VIRTUAL ENVIRONMENT

A **virtual environment** ("venv") is a private, throwaway copy of Python that lives inside your project folder. Packages you install go *there*, not into your system Python. This stops different projects from fighting over package versions — essential because LangChain pulls in many dependencies.

Pick a folder for the whole course and create the venv inside it:

```
# 1. make a project folder and move into it
mkdir langchain-course
cd langchain-course

# 2. create a virtual environment named ".venv"
python3 -m venv .venv
```

```
# 3. activate it (this is the step people forget)
# macOS / Linux:
source .venv/bin/activate
# Windows PowerShell:
.venv\Scripts\Activate.ps1
```

Once activated, your prompt shows `(.venv)` at the start of the line. That's how you know packages will land in the right place. You must re-activate every time you open a new terminal.

LINE BY LINE

- `mkdir langchain-course` — "make directory": creates the folder.
- `cd langchain-course` — "change directory": steps into it.
- `python3 -m venv .venv` — runs Python's built-in `venv` module (`-m` = "run this module") to create a venv in a hidden folder called `.venv` .
- `source .venv/bin/activate` — turns the venv on for this terminal session.

0.4 — Install the course packages

With the venv active, install everything we'll use across all 13 phases in one go:

```
pip install -U \
  langchain \
  langchain-google-genai \
  langchain-ollama \
  langchain-community \
  langgraph \
  faiss-cpu \
  pypdf \
  python-dotenv
```

(On Windows PowerShell, put it all on one line and remove the `\` characters.)

Package	What it gives you	First used in
<code>langchain</code>	The main library. Pulls in <code>langchain-core</code> (the Runnable interface, messages, prompts) automatically.	Phase 1
<code>langchain-google-genai</code>	The Gemini integration: <code>ChatGoogleGenerativeAI</code> and <code>GoogleGenerativeAIEmbeddings</code> .	Phase 2
<code>langchain-ollama</code>	The local integration: <code>ChatOllama</code> and <code>OllamaEmbeddings</code> .	Phase 2
<code>langchain-community</code>	Community-maintained integrations: document loaders, the FAISS vector store wrapper, etc.	Phase 6
<code>langgraph</code>		Phase 5

	Stateful graphs. Underpins modern memory (Phase 5) and agents (Phase 9), and is the focus of Phase 10.	
<code>faiss-cpu</code>	A fast local vector database for storing embeddings. Free, runs on CPU.	Phase 7
<code>pypdf</code>	Lets the PDF document loader read <code>.pdf</code> files.	Phase 6
<code>python-dotenv</code>	Loads secrets from a <code>.env</code> file into your program. Keeps your API key out of your code.	Phase 0 (next)

VERIFY THE INSTALL

Run this one-liner; it should print a version number with no errors:

```
python3 -c "import langchain; print(langchain.__version__)"
```

You'll see something like `1.x.x`.

0.5 — Get a free Google Gemini API key

Gemini is our cloud model. Google gives every account a free tier that is plenty for learning.

1. Go to aistudio.google.com (Google AI Studio) and sign in with a Google account.
2. Click **"Get API key"** (left sidebar or top-right).
3. Click **"Create API key"**. Copy the long string it gives you — it starts with `AIza...`.
4. Treat it like a password. We'll store it safely in step 0.7 — **do not paste it directly into your code**.

PROTECT YOUR KEY

Never commit an API key to GitHub, never put it in a screenshot, never share it in a chat. If it leaks, return to AI Studio and delete/regenerate it immediately. A leaked key can run up charges on your account.

WHICH MODEL WILL WE CALL?

Throughout the course we'll use `gemini-2.5-flash` — fast, cheap/free-tier-friendly, and very capable for learning. Newer models (the Gemini 3 family) exist and you can swap the name in later, but `gemini-2.5-flash` is the reliable default. You can always see the current model list inside AI Studio.

0.6 — Install Ollama and pull local models

Ollama is a small program that downloads open-weight models and runs them on your own machine, exposing them through a local server.

1. Download and install Ollama for your OS from ollama.com/download (macOS, Windows, and Linux all supported; on Linux/WSL you can also use the install script on that page).
2. After installing, Ollama runs a background server at `http://localhost:11434`. You don't manage it directly — LangChain will talk to it.
3. Open a terminal and pull the models this course uses:

```
# a small, capable chat model (~2 GB download)
ollama pull llama3.2

# Google's open model, as a second chat option
ollama pull gemma3

# a dedicated EMBEDDING model for RAG (Phases 7-8)
ollama pull nomic-embed-text
```

Check what you've got:

```
ollama list
```

Expected output (sizes/IDs will differ):

NAME	ID	SIZE	MODIFIED
llama3.2:latest	a80c4f17acd5	2.0 GB	10 seconds ago
gemma3:latest	a2af6cc3eb7f	3.3 GB	1 minute ago
nomic-embed-text:latest	0a109f422b47	274 MB	2 minutes ago

TERM: EMBEDDING MODEL

An **embedding** turns a piece of text into a list of numbers that captures its meaning, so a computer can measure how similar two texts are. `nomic-embed-text` is built only for that job (it doesn't chat). We'll use it heavily in Phases 7–8 for search and RAG.

HARDWARE REALITY CHECK

Local models use RAM and are slower than the cloud. `llama3.2` and `gemma3` run on most laptops with 8 GB+ RAM, but expect a few seconds per answer on a CPU-only machine. That's normal — it's the trade-off for free, private, offline inference. If a model is too slow, try a smaller tag like `ollama pull llama3.2:1b`.

0.7 — Store secrets safely in a `.env` file

TERM: ENVIRONMENT VARIABLE & .ENV FILE

An **environment variable** is a named value your operating system hands to programs (e.g. `GOOGLE_API_KEY`). A **.env file** is a plain text file holding these as `NAME=value` lines. The `python-dotenv` package reads it and loads the values, so your secret lives in one file you never share — not scattered through your code.

Inside your `langchain-course` folder, create a file named exactly `.env` (the leading dot matters) with this single line, pasting *your* key:

```
GOOGLE_API_KEY=AIzaSyD...your_real_key_here
```

Then create a file named `.gitignore` so the key can never be committed to version control:

```
.env
.venv/
```

WHY `GOOGLE_API_KEY` SPECIFICALLY?

`langchain-google-genai` automatically looks for an environment variable named `GOOGLE_API_KEY`. If it finds it, you never have to mention the key in your code at all. (The underlying Google SDK also accepts `GEMINI_API_KEY` — either works, but we'll standardise on `GOOGLE_API_KEY`.)

0.8 — Smoke test: prove Gemini works

Create a file called `test_gemini.py` in your project folder:

```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()                                # 1. read .env into environment variables

llm = ChatGoogleGenerativeAI(                 # 2. create the model object
    model="gemini-2.5-flash",                 # 3. choose which Gemini model
    temperature=0,                           # 4. 0 = focused/repeatable answers
)

response = llm.invoke("Say hello in exactly five words.") # 5. call the model

print(response.content)                       # 6. print just the text
```

Line-by-line

Line	What it does
1	<code>load_dotenv()</code> reads your <code>.env</code> file and loads <code>GOOGLE_API_KEY</code> so the library can find it. Call this once, early.
2–4	Builds a <code>ChatGoogleGenerativeAI</code> object — your handle to Gemini. <code>model</code> picks the model; <code>temperature=0</code> makes answers as deterministic as possible (we cover temperature in Phase 2).
5	<code>.invoke(...)</code> sends your text and waits for the full reply. This is the single most important method in LangChain — every model and chain has it.
6	The reply is a message <i>object</i> ; <code>.content</code> pulls out the plain text. (Printing <code>response</code> alone shows lots of extra metadata.)

Run it:

```
python3 test_gemini.py
```

Expected output (the exact words vary):

```
Hello, nice to meet you!
```

IF YOU SEE FIVE WORDS BACK — YOUR CLOUD SETUP WORKS.

You just ran inference on a model sitting in Google's data centre, authenticated with your key, all through three lines of LangChain.

0.9 — Smoke test: prove Ollama works

Create `test_ollama.py`. Note how similar it is — that's LangChain's whole point.

```
from langchain_ollama import ChatOllama

llm = ChatOllama(
    model="llama3.2",          # 1. point at a LOCAL model — no API key needed
    temperature=0,           # 2. must match a name from `ollama list`
)

response = llm.invoke("Say hello in exactly five words.") # 3. same method as Gemini
print(response.content)
```

Line-by-line

Line	What it does
1	<code>ChatOllama</code> talks to the Ollama server on your machine. No <code>load_dotenv</code> , no key — it's all local.
2	<code>model="llama3.2"</code> must exactly match a model you pulled. Names are case-sensitive: <code>Llama3.2</code> will fail.
3	The exact same <code>.invoke(...)</code> call and the exact same <code>.content</code> result as Gemini. You could swap the two model objects and the rest of your program wouldn't change.

Run it:

```
python3 test_ollama.py
```

Expected output (first run is slower while the model loads into memory):

```
Hi there, how are you?
```

IF YOU SEE A REPLY — YOUR LOCAL SETUP WORKS TOO.

You now have both backends running. Everything from Phase 1 onward will be shown in both flavours, and you've just proven they behave the same.

0.10 — Common errors & troubleshooting

Symptom	Likely cause & fix
<code>ModuleNotFoundError: No module named 'langchain_google_genai'</code>	The venv isn't active, or you installed into a different Python. Re-run <code>source .venv/bin/activate</code> (you should see <code>(.venv)</code>), then re-run the <code>pip install</code> command.
<code>DefaultCredentialsError</code> / "API key not found" / 400 from Gemini	The key isn't loaded. Confirm: (a) the file is named <code>.env</code> (not <code>.env.txt</code>), (b) the line is <code>GOOGLE_API_KEY=...</code> with no quotes/spaces, (c) you called <code>load_dotenv()</code> before creating the model, (d) you're running the script from inside the project folder.
Gemini: <code>404 ... model not found</code>	The model name is wrong or retired. Use <code>gemini-2.5-flash</code> . Check AI Studio for the current list.
Ollama: <code>ConnectionError</code> / "connection refused" to <code>localhost:11434</code>	The Ollama server isn't running. Open the Ollama app (macOS/Windows), or run <code>ollama serve</code> in a terminal (Linux), then try again.
Ollama: <code>model "llama3.2" not found, try pulling it first</code>	You didn't pull it, or mistyped the name. Run <code>ollama pull llama3.2</code> and verify with <code>ollama list</code> . Names are case-sensitive.
Ollama answers are painfully slow	Normal on CPU-only laptops. Use a smaller model: <code>ollama pull llama3.2:1b</code> and set <code>model="llama3.2:1b"</code> .
Windows: <code>Activate.ps1 cannot be loaded ... execution policy</code>	Run once in PowerShell: <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , then activate again.

0.11 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 1)

Task: Make one new file, `test_both.py`, that asks *both* models the same question — `"In one sentence, what is a token in an LLM?"` — and prints each answer under a label, like:

```
== Gemini ==
<its answer>

== Ollama (llama3.2) ==
<its answer>
```

Why: it forces you to combine both smoke tests, confirms both backends work side by side, and previews how easily LangChain lets you swap models — the foundation for the rest of the course.

Hints (not the full answer):

- You only need `load_dotenv()` once, at the top, even though only Gemini uses it.
- Import *both* `ChatGoogleGenerativeAI` and `ChatOllama` and make two model objects with different variable names.

- The question text is identical for both — store it in one variable and pass it to each `.invoke()`.
- Use plain `print("== Gemini ==")` lines as labels between the two answers.

Terms introduced this phase

Term	One-line meaning
LLM	A model that takes in text and predicts useful text back.
Token	The chunk an LLM reads/writes; ~¾ of a word.
Inference	Running a model to get an answer ("calling the model").
API key	Secret string proving you may use a cloud service; sent with each request.
Cloud model	Runs on a provider's servers; needs internet + key (e.g. Gemini).
Local model	Runs on your machine; free, private, offline (e.g. via Ollama).
Virtual environment (venv)	An isolated per-project copy of Python for installing packages safely.
Embedding	Text turned into numbers that capture meaning; powers search & RAG.
Environment variable / .env	Named values handed to programs; <code>.env</code> stores secrets out of your code.
<code>.invoke()</code>	The core "run it once and give me the result" method on every model and chain.

PHASE 0 COMPLETE

You have an isolated Python environment, all packages installed, a working Gemini key, local models running under Ollama, secrets safely in `.env`, and proof that both backends respond. **Next up — Phase 1: LangChain Fundamentals**, where we explain what LangChain actually is, meet the Runnable interface, and use the `|` pipe operator to build your first chain.